



LJ University
University with a Difference

DATA STRUCTURE USING JAVA

LECTURE NOTES FOR UNIT - 2

UNIT -2 STACK - I

PREPARED BY:
MR. VISMAY SHAH

Contents

2.1	Stack	2
a.	Some key points related to stack	3
b.	Working of Stack	3
2.2	Stack Operations	4
a.	PUSH OPERATION	5
b.	POP OPERATION	7
c.	PEEP OPERATION	9
d.	CHANGE OPERATION	10
2.3	JAVA Programs for Stack Operations	11
2.4	Application of Stack	19
a.	Balancing of Symbols	19
b.	String Reversal	19
c.	Undo/ Redo	20
d.	Recursion	20
e.	DFS	20
f.	Backtracking	20
g.	Expression Conversion	20
h.	Memory Management	21
2.5	Recursion	21
a.	Difference Between Recursion and Iteration: -	22
2.6	Recursive Method	23
a.	Memory allocation of Recursive method	24
2.7	Advantages and Disadvantages of Recursion	25
2.8	Tower of Hanoi Problem Using Recursion	26
a.	JAVA PROGRAM FOR TOWER HANOI	27
2.9	String Reversal Using Stack	29
b.	JAVA PROGRAM FOR STRING REVERSAL	29
2.10	Find Binary of The Given Decimal Number Using Stack	30
c.	JAVA PROGRAM TO FIND BINARY FROM DECIMAL	30

2.1 Stack

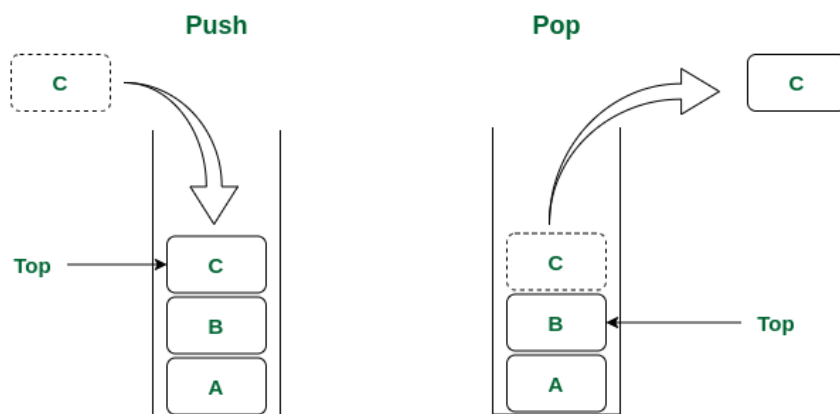
A stack is a linear list of data elements in which insertion of new element and deletion of the element occurs at the one end. This end is called as **TOP** of the STACK.

Stack is a linear data structure that follows a particular order in which the operations are performed.

The order may be **LIFO (Last In First Out)** or **FILO (First In Last Out)**. LIFO implies that the element that is inserted last, comes out first and FILO implies that the element that is inserted first, comes out last.

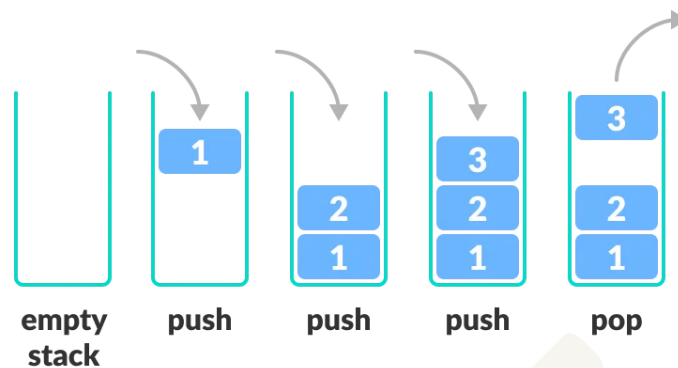
The operation of inserting the new element in the stack and the deletion of the element from the stack is called as **PUSH & POP** Respectively.

It is a linear list in which insertion and deletion take place only at one end. It is used to evaluate different expressions.



Stack Data Structure

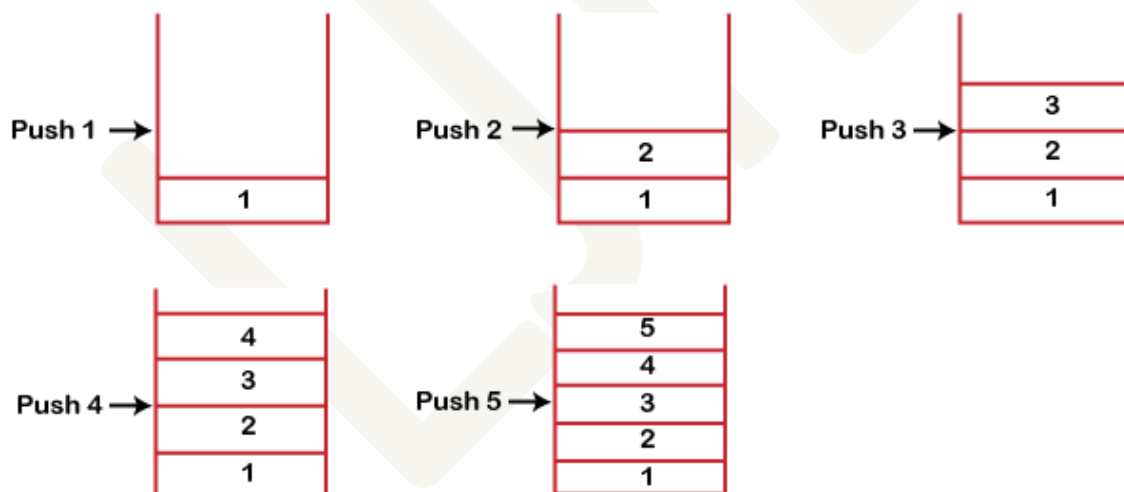
There are many real-life examples of a stack. Consider an example of plates stacked over one another in the canteen. The plate which is at the top is the first one to be removed, i.e. the plate which has been placed at the bottommost position remains in the stack for the longest period of time. So, it can be simply seen to follow LIFO (Last In First Out)/FILO (First In Last Out) order.



a. Some key points related to stack

- ✓ It is called as stack because it behaves like a real-world stack, piles of books, etc.
- ✓ A Stack is an abstract data type with a pre-defined capacity, which means that it can store the elements of a limited size.
- ✓ It is a data structure that follows some order to insert and delete the elements, and that order can be LIFO or FILO

b. Working of Stack



- As we can observe in the above figure there are five memory blocks in the stack; therefore, the size of the stack is 5.
- Suppose we want to store the elements in a stack and let's assume that stack is empty.
- We have taken the stack of size 5 as shown below in which we are pushing the elements one by one until the stack becomes full.
- In the above cases, we can observe that it goes from the bottom to the top when we were entering the new element in the stack.
- When we perform the delete operation on the stack, there is only one way for entry and exit as the other end is closed.
- It follows the LIFO pattern, which means that the value entered Last will be removed First.
- In the above case, the value 5 is entered Last, so it will be removed First and similarly 1 was entered First will be removed Last.

2.2 Stack Operations

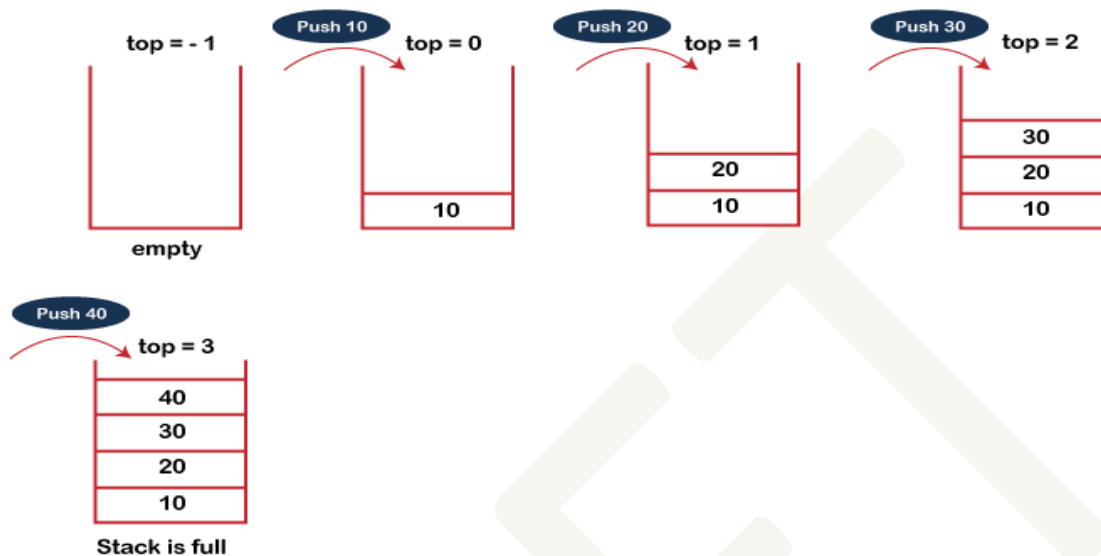
The following are some common operations implemented on the stack:

1. **push():** When we insert an element in a stack then the operation is known as a push. If the stack is full then the overflow condition occurs.
2. **pop():** When we delete an element from the stack, the operation is known as a pop. If the stack is empty means that no element exists in the stack, this state is known as an underflow state.
3. **isEmpty():** It determines whether the stack is empty or not.
4. **isFull():** It determines whether the stack is full or not.'
5. **peek():** Fetch the element from the ith index of the Stack. (i.e. it gives us element from the ith index from the TOP of the stack).
6. **count():** It returns the total number of elements available in a stack.
7. **change():** It changes the element at the ith index from the TOP of the Stack
8. **display():** It prints all the elements available in the stack.

Note:-Remember that - All operations on the stack are performed from the TOP of the Stack.

a. PUSH OPERATION

The steps involved in the PUSH operation is given below:



- ▶ Before inserting an element in a stack, we must check whether the stack is full or not?
- ▶ If we try to insert the element in a stack, and the stack is full, then the overflow condition occurs.
- ▶ When we initialize a stack, we set the value of **top as -1** to check that the stack is empty.
- ▶ When the new element is pushed in a stack, first, the value of the top gets incremented, i.e., **top=top+1**, and the element will be placed at the new position of the top.
- ▶ The elements will be inserted until we reach the max size of the stack.

PUSH OPERATION – ALGORITHM

Procedure PUSH (S, TOP, X)

This procedure inserts an element X to the top of a stack which is represented by a vector S containing N elements with a pointer TOP denoting the top element in the stack.

STEP 1: [Check for stack overflow?]

 If $TOP \geq N$
 then Write ("STACK is Overflow")

 Return

STEP 2: [Increment TOP]

$TOP \leftarrow TOP + 1$

STEP 3: [Insert Element]

$S[TOP] \leftarrow X$

STEP 4: [Finished]

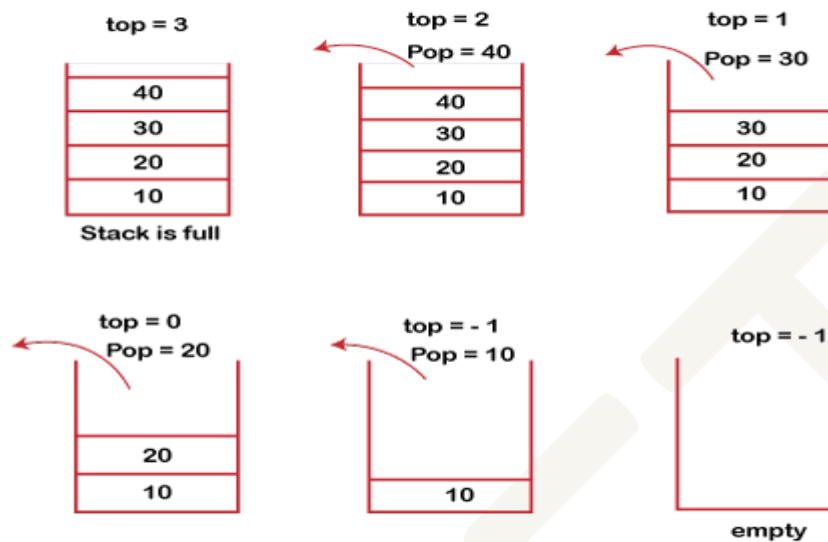
 Return

Small Description: Here, in this algorithm in the first step we are checking the overflow condition for the stack. Here in the stack the maximum size of the stack is N. So that if the stack's top pointer is equal to or greater than N then its overflow and operation was completed there.

Now, if condition gets false then top pointer is incremented by one. And we assign the new value X at that position.

b. POP OPERATION

The steps involved in the POP operation is given below:



- Before deleting the element from the stack, we need to check whether the stack is empty or not.
- If we try to delete the element from the empty stack, then the **underflow condition occurs**.
- If the stack is not empty, we first access the element which is pointed by the top
- Once the pop operation is performed, the top is decremented by 1, i.e., $top = top - 1$.

POP OPERATION – ALGORITHM**Method POP(S, TOP)**

This method removes the top element from the stack which is represented by a vector S and returns this element. TOP is a pointer to the top element of the stack.

Step 1: [Check for the Underflow on stack]

 If TOP = 0
 then Write('STACK UNDERFLOW ON POP')
 take action in response to underflow
 Exit

Step 2: [Decrement top Pointer]

 TOP ← TOP-1

Step 3: [Returns pointer top element of stack]

 Return(S[TOP + 1])

Small Description: Here in this method first we check for the underflow condition because if top is pointing to the zero means there is no element on the stack. Now if the condition got wrong then for deleting the elements from the stack first decrement the top pointer. And then return the element on the index top+1. So we write return S[TOP+1].

c. PEEP OPERATION

PEEP OPERATION – ALGORITHM

Method PEEP(S, TOP, I)

In this method the S is a vector (consisting the N elements) representing a sequential allocated stack, and a pointer TOP denoting the top element of the Stack, this method returns the value of the Ith element from the TOP of the stack. The element is not deleted by this method.

Step 1: [Check for the Stack Underflow]

If $TOP - I + 1 \leq 0$

then Write("Stack Underflow On Peep")
take action in response to underflow

Exit

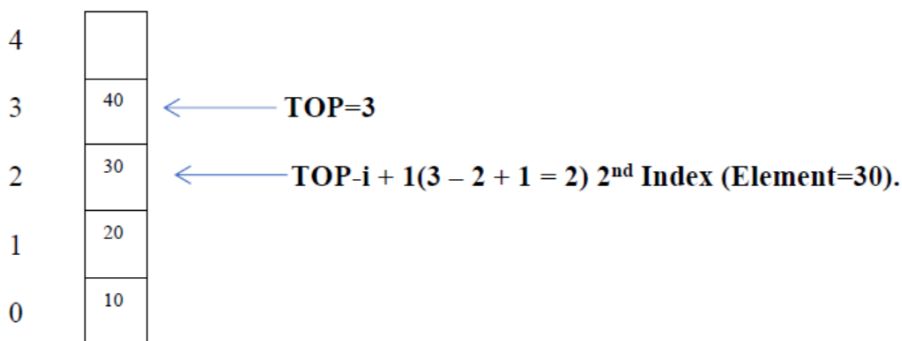
Step 2: [Return Ith Element from the top of the Stack]

Return(S [TOP – I + 1])

Small Description: - Here, we first check the underflow of the stack. Here we check the $TOP - I + 1 \leq 0$, because 'I' is the index from which the element is fetched out, and the index 'I' is calculated from the TOP.

For Example if the $I=3$, the stack size is 5 and the top pointer is at position 5 then the actual index for the fetching the element is $TOP - I + 1 = 5 - 3 + 1 = 3$. So the 3rd element from the top pointer is fetched out. So that if any element exists on that 'TOP-I+1' index then the element is fetched out successfully, otherwise the error message of Underflow prints.

If index is 3 from the TOP then, in below figure the TOP is at index -3. But we want element from the 2nd index of the TOP. So it will become $TOP - I + 1 = 3 - 2 + 1 = 2$ (Index). Because from the TOP the 2nd element is = 30, which is @ 2nd index of the stack. Here, $I=2$. So the Answer = 30.



d. CHANGE OPERATION

CHANGE OPERATION – ALGORITHM

Procedure CHANGE(S, TOP, I, X)

In this procedure the vector 'S' represents a sequentially allocated stack and a pointer TOP denotes the top element of the stack. This procedure changes the value of the Ith element from the top of the stack to the value contained in X.

Step 1: [Check for the Stack Underflow]

If $TOP - I + 1 \leq 0$
then Write("Stack Underflow On CHANGE")
Return

Step 2: [Change Ith Element from the top of the Stack]

$S[TOP - I + 1] \leftarrow X$

Step 3: [Finished]

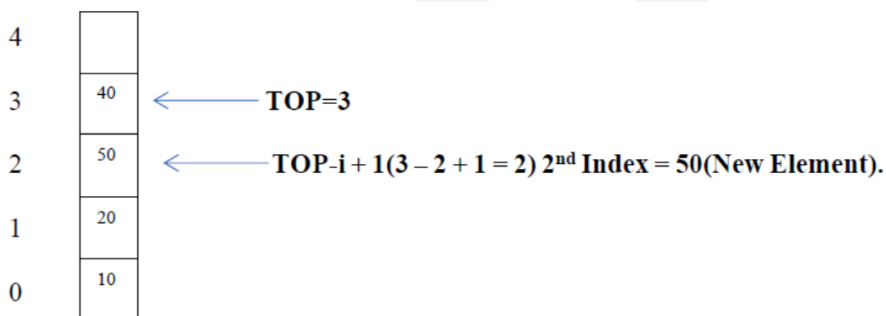
Return

Small Description: - Here, we first check the underflow of the stack. Here we check the $TOP - I + 1 \leq 0$, because 'I' is the index on which the element is going to be changed and new value X is inserted at that index, and the index 'I' is calculated from the TOP.

For Example if the $I=3$, the stack size is 5, New value $X=10$ and the top pointer is at position 4 then the actual index for the change the element is $TOP - I + 1 = 4 - 3 + 1 = 2$. So the new value 10 is inserted at the 3rd element from the top pointer (which becomes the 2nd index of the stack). So that if any element exists on that 'TOP-I+1' index then the element is changed successfully, otherwise the error message of Underflow prints.

Because from the TOP the 2nd element is = 30, which is @ 2nd index of the stack and new element = 50 so that.

Here, $I=2$. So the Answer = 50.



2.3 JAVA Programs for Stack Operations

Write A Program to PUSH elements in a Stack (Where Stack size is taken from user) and display the inserted elements. Also, check the overflow condition.

```
1  import java.util.*;
2  class Stacks
3  {
4      static int top; int cap; //declared two variables top and cap//
5      int a[]; // declared an Array//
6      Stack(int size)//created a paramterized constructor//
7      {
8          a=new int[size]; //Array a will be now declared with the size entered by user//
9          System.out.println("Your Stack size is "+size);
10         cap=size; //Stack capacity will be same equal to size entered by user//
11         top=-1; //top will be initialized with top equal to -1//
12     }
13     void push(int data) //created a method to push a data into Stack//
14     {
15         if(top==cap-1) //checking Overflow condition//
16         {
17             System.out.println("overflow");
18         }
19         else //otherwise//
20         {
21             top++; //top pointer shall be incremented//
22             a[top]=data; //the data will be placed in the Array a at TOP index//
23             System.out.println("element enter sucessfully");
24         }
25     }
26     void display() //Created a method to display the stack elements//
27     {
28         for(int i=top; i>=0; i--) //used the loop to print all stack//
29             System.out.println(a[i]);
30     }
31 }
32 class main
33 {
34     public static void main(String[] args)
35     {
36         System.out.print("Enter A Stack Size ");
37         Scanner sc=new Scanner(System.in);
38         int n=sc.nextInt();
39         Stack s1=new Stack(n);
40         s1.push(11);
41         s1.push(12);
42         s1.push(13);
43         s1.push(14);
44         s1.push(15);
45         s1.display();
46     }
47 }
```

Write A Program to PUSH & POP elements in a Stack (Where Stack size is taken from user) and display the stack after PUSH & POP Operation. Also, check the overflow condition.

```
1  import java.util.*;
2  class Stacks
3  {
4      static int top; int cap; //declared two variables top and cap//
5      int a[]; // declared an Array//
6      Stack(int size)//created a parameterized constructor//
7      {
8          a=new int[size];//Array a will be now declared with the size entered by user//
9          System.out.println("Your Stack size is "+size);
10         cap=size; //Stack capacity will be same equal to size entered by user//
11         top=-1; //top will be initialized with top equal to -1//
12     }
13     void push(int data) //created a method to push a data into Stack//
14     {
15         if(top==cap-1) //checking Overflow condition//
16         {
17             System.out.println("overflow");
18         }
19         else //otherwise//
20         {
21             top++; //top pointer shall be incremented//
22             a[top]=data;//the data will be placed in the Array a at TOP index//
23             System.out.println("element enter sucessfully");
24         }
25     }
26     void pop() //created a method to pop a data into Stack//
27     {
28         if(top== -1) //Checking for Underflow Condition//
29         {
30             System.out.println("underflow");
31         }
32         else //otherwise//
33         {
34             System.out.println(a[top]+ "is deleted");
35             top--; //top pointer decreases//
36         }
37     }
38     void display() //Created a method to display the stack elements//
39     {
40         for(int i=top;i>=0;i--) //used the loop to print all stack//
41             System.out.println(a[i]);
42     }
43 }
44 class Run
45 {
46     public static void main(String[] args)
47     {
48         System.out.print("Enter A Stack Size ");
49         Scanner sc=new Scanner(System.in);
50         int n=sc.nextInt();
51         Stack s1=new Stack(n);
52         s1.push(11);
53         s1.push(12);
54         s1.push(13);
55         s1.push(14);
56         s1.push(15);
57         s1.display();
58         s1.pop();
59         s1.display();
60         s1.pop();
61         s1.display();
62     }
63 }
```

Write A Program to PUSH, POP & PEEP elements in a Stack (Where Stack size is taken from user) and display the stack after PUSH, POP & PEEP Operation. Also, check the overflow & Underflow condition.



DATA STRUCTURES (017013292)

Semester – II

Chapter Name: STACK-1

```
1  import java.util.*;
2  class Stacks
3  {
4      static int top; int cap; //declared two variables top and cap//
5      int a[]; // declared an Array//
6      Stack(int size)//created a paramterized constructor//
7      {
8          a=new int[size];//Array a will be now declared with the size entered by user//
9          System.out.println("Your Stack size is "+size);
10         cap=size; //Stack capacity will be same equal to size entered by user//
11         top=-1; //top will be initialized with top equal to -1//
12     }
13     void push(int data) //created a method to push a data into Stack//
14     {
15         if(top==cap-1) //checking Overflow condition//
16         {
17             System.out.println("overflow");
18         }
19         else //otherwise//
20         {
21             top++; //top pointer shall be incremented//
22             a[top]=data;//the data will be placed in the Array a at TOP index//
23             System.out.println(a[top]+" is enter sucessfully");
24         }
25     }
26     void pop() //created a method to pop a data into Stack//
27     {
28         if(top==--1) //Checking for Underflow Condition//
29         {
30             System.out.println("underflow");
31         }
32         else //otherwise//
33         {
34             System.out.println(a[top]+" is deleted");
35             top--; //top pointer decreases//
36         }
37     }
38     void display() //Created a method to display the stack elements//
39     {
40         for(int i=top;i>=0;i--) //used the loop to print all stack//
41             System.out.println(a[i]);
42     }
43     void peep(int i) //Created a Method to Check the value of ith elment from TOP//
44     {
45         if(top-i+1<=-1) //Check for Underflow condition//
46         {
47             System.out.println("underflow");
48         }
49         else //otherwise//
50         {
51             System.out.println(a[top-i+1]+"is at position " +i);
52         }
53     }
54 }
55
56 class Run
57 {
58     public static void main(String[] args)
59     {
60         System.out.print("Enter A Stack Size ");
61         Scanner sc=new Scanner(System.in);
62         int n=sc.nextInt();
63         Stack s1=new Stack(n);
64         s1.push(11);
65         s1.push(12);
66         s1.push(13);
67         s1.push(14);
```

```
68     s1.push(15);
69     s1.display();
70     s1.pop();
71     s1.display();
72     s1.pop();
73     s1.display();
74     s1.peep(2);
75     s1.display();
76 }
77 }
```


Write A Program to Change the Element of Stack (Where Stack size is taken from User) and also display original stack and updated stack. Also, check the overflow and underflow conditions.



DATA STRUCTURES (017013292)

Semester – II

Chapter Name: STACK-1

```
1  import java.util.*;
2  class Stacks
3  {
4      static int top; int cap; //declared two variables top and cap//
5      int a[]; // declared an Array//
6      Stack(int size)//created a paramterized constructor//
7      {
8          a=new int[size];//Array a will be now declared with the size entered by user//
9          System.out.println("Your Stack size is "+size);
10         cap=size; //Stack capacity will be same equal to size entered by user//
11         top=-1; //top will be initialized with top equal to -1//
12     }
13     void push(int data) //created a method to push a data into Stack//
14     {
15         if(top==cap-1) //checking Overflow condition//
16         {
17             System.out.println("overflow");
18         }
19         else //otherwise//
20         {
21             top++; //top pointer shall be incremented//
22             a[top]=data;//the data will be placed in the Array a at TOP index//
23             System.out.println(a[top]+" is enter sucessfully");
24         }
25     }
26     void pop() //created a method to pop a data into Stack//
27     {
28         if(top== -1) //Checking for Underflow Condition//
29         {
30             System.out.println("underflow");
31         }
32         else //otherwise//
33         {
34             System.out.println(a[top]+" is deleted");
35             top--; //top pointer decreases//
36         }
37     }
38     void display() //Created a method to display the stack elements//
39     {
40         for(int i=top;i>=0;i--) //used the loop to print all stack//
41             System.out.println(a[i]);
42     }
43     void peep(int i) //Created a Method to Check the value of ith elment from TOP//
44     {
45         if(top-i+1<=-1) //Check for Underflow condition//
46         {
47             System.out.println("underflow");
48         }
49         else //otherwise//
50         {
51             System.out.println(a[top-i+1]+"is at position " +i);
52         }
53     }
54     void change(int x,int i) //Created a method to insert X in place of i//
55     {
56         if(top-i+1<=-1) //Check for underflow condition//
57         {
58             System.out.println("underflow");
59         }
60         else //otherwise//
61         {
62             a[top-i+1] = x; //new element x will be take place at ith position from top//
63         }
64     }
65 }
66 class Run
67 {
```

```
68     public static void main(String arg[])
69     {
70         System.out.print("Enter A Stack Size ");
71         Scanner sc=new Scanner(System.in);
72         int n=sc.nextInt();
73         Stack s1=new Stack(n);
74         s1.push(11);
75         s1.push(12);
76         s1.push(13);
77         s1.push(14);
78         s1.push(15);
79         s1.display();
80         s1.change(120,4);
81         s1.display();
82     }
83 }
```

2.4 Application of Stack

The following are the applications of the stack:

a. Balancing of Symbols

Balancing of symbols: Stack is used for balancing a symbol. For example, we have the following program:

```
class Run
{
    public static void main(String args[])
    {System.out.println("Hello");}
}
```

As we know, each program has an opening and closing braces; when the opening braces come, we push the braces in a stack, and when the closing braces appear, we pop the opening braces from the stack. Therefore, the net value comes out to be zero. If any symbol is left in the stack, it means that some syntax occurs in a program.

b. String Reversal

String reversal: Stack is also used for reversing a string.

For example, we want to reverse a "Intro Stack" string, so we can achieve this with the help of a stack. First, we push all the characters of the string in a stack until we reach the null character.

After pushing all the characters, we start taking out the character one by one until we reach the bottom of the stack.

c. Undo/ Redo

UNDO/REDO: It can also be used for performing UNDO/REDO operations.

For example, we have an editor in which we write 'a', then 'b', and then 'c'; therefore, the text written in an editor is abc. So, there are three states, a, ab, and abc, which are stored in a stack. There would be two stacks in which one stack shows UNDO state, and the other shows REDO state.

If we want to perform UNDO operation, and want to achieve 'ab' state, then we implement pop operation.

d. Recursion

Recursion: The recursion means that the method is calling itself again. To maintain the previous states, the compiler creates a system stack in which all the previous records of the method are maintained.

e. DFS

DFS (Depth First Search): This search is implemented on a Graph, and Graph uses the stack data structure.

f. Backtracking

Backtracking: Suppose we have to create a path to solve a maze problem. If we are moving in a particular path, and we realize that we come on the wrong way. In order to come at the beginning of the path to create a new path, we have to use the stack data structure.

g. Expression Conversion

Expression conversion: Stack can also be used for expression conversion. This is one of the most important applications of stack. The list of the expression conversion is given below:

- Infix to prefix
- Infix to postfix
- Prefix to infix
- Prefix to postfix
- Postfix to infix

h. Memory Management

Memory management: The stack manages the memory. The memory is assigned in the contiguous memory blocks. The memory is known as stack memory as all the variables are assigned in a method call stack memory. The memory size assigned to the program is known to the compiler. When the method is created, all its variables are assigned in the stack memory. When the method completed its execution, all the variables assigned in the stack are released.

2.5 Recursion

- ▶ Recursion is the process which comes into existence when a method calls a copy of itself to work on a smaller problem. Any method which calls itself is called recursive method, and such method calls are called recursive calls.
- ▶ Recursion involves several numbers of recursive calls. However, it is important to impose a termination condition of recursion. Recursion code is shorter than iterative code; however it is difficult to understand.
- ▶ Recursion cannot be applied to all the problem, but it is more useful for the tasks that can be defined in terms of similar subtasks. For Example, recursion may be applied to sorting, searching, and traversal problems.
- ▶ Generally, iterative solutions are more efficient than recursion since method call is always overhead. Any problem that can be solved recursively, can also be solved iteratively. However, some problems are best suited to be solved by the recursion, for example, tower of Hanoi, Fibonacci series, factorial finding, etc.

a. Difference Between Recursion and Iteration: -

Property	Recursion	Iteration
Definition	Method calls itself.	A set of instructions repeatedly executed.
Application	For methods.	For loops.
Termination	Through base case, where there will be no method call.	When the termination condition for the iterator ceases to be satisfied.
Usage	Used when code size needs to be small, and time complexity is not an issue.	Used when time complexity needs to be balanced against an expanded code size.
Code Size	Smaller code size	Larger Code Size.
Time Complexity	Very high (generally exponential) time complexity.	Relatively lower time complexity (generally polynomial-logarithmic).

In the following example, recursion is used to calculate the factorial of a number.

return 5 * factorial(4) = 120

└─ return 4 * factorial(3) = 24

└─ return 3 * factorial(2) = 6

└─ return 2 * factorial(1) = 2

└─ return 1 * factorial(0) = 1

javaTpoint.com

1 * 2 * 3 * 4 * 5 = 120

2.6 Recursive Method

Recursive Method

- ▶ A recursive method performs the tasks by dividing it into the subtasks. There is a termination condition defined in the method which is satisfied by some specific subtask. After this, the recursion stops and the final result is returned from the method.
- ▶ The case at which the method doesn't recur is called the base case whereas the instances where the method keeps calling itself to perform a subtask, is called the recursive case. All the recursive methods can be written using this format.

Pseudocode for writing any recursive method is given below.

```
if (test_for_base)
{
    return some_value;
}

else if (test_for_another_base)
{
    return some_another_value;
}

else
{
    // Statements;

    recursive call;
}
```


a. Memory allocation of Recursive method

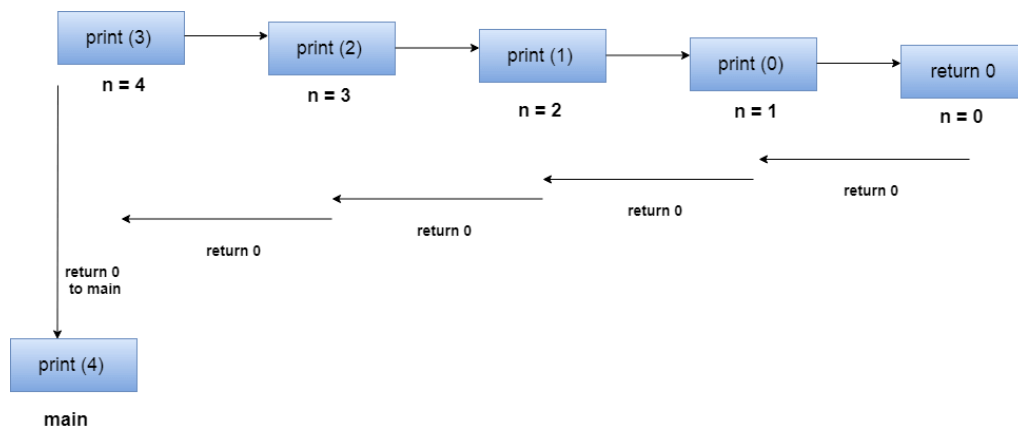
- ▶ Each recursive call creates a new copy of that method in the memory. Once some data is returned by the method, the copy is removed from the memory. Since all the variables and other stuff declared inside method get stored in the stack, therefore a separate stack is maintained at each recursive call.
- ▶ Once the value is returned from the corresponding method, the stack gets destroyed. Recursion involves so much complexity in resolving and tracking the values at each recursive call. Therefore we need to maintain the stack and track the values of the variables defined in the stack.
- ▶ Let us consider the following example to understand the memory allocation of the recursive methods.

```
int display (int n)
{
    if(n == 0)

        return 0; // terminating condition else
    {
        System.out.println(n);
        return display(n-1); // recursive call
    }
}
```

Explanation:

Let us examine this recursive method for $n = 4$. First, all the stacks are maintained which prints the corresponding value of n until n becomes 0, Once the termination condition is reached, the stacks get destroyed one by one by returning 0 to its calling stack. Consider the following image for more information regarding the stack trace for the recursive methods.



Stack tracing for recursive function call

2.7 Advantages and Disadvantages of Recursion

Advantages of Recursion

1. The code may be easier to write.
2. Reduce unnecessary calling of method.
3. Extremely useful when applying the same solution.
4. Recursion reduce the length of code.
5. It is very useful in solving the data structure problem.
6. Stacks evolutions and infix, prefix, postfix evaluations etc.

Disadvantages of Recursion

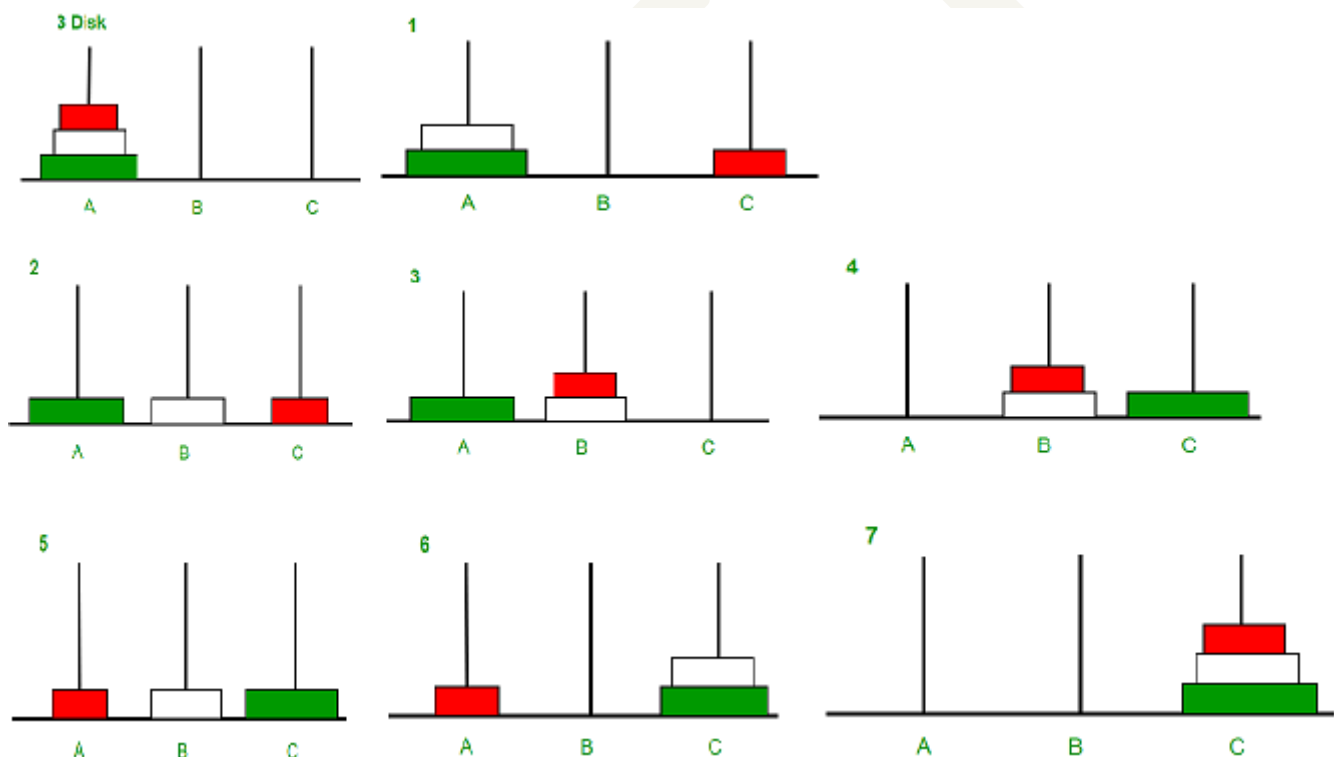
1. Recursive methods are generally slower than non-recursive method.
2. It may require a lot of memory space to hold intermediate results on the system stacks.
3. Hard to analyze or understand the code.
4. It is not more efficient in terms of space and time complexity.
5. The computer may run out of memory if the recursive calls are not properly checked.

2.8 Tower of Hanoi Problem Using Recursion

Tower of Hanoi Problem using Recursion

Tower of Hanoi is a mathematical puzzle where we have three rods and n disks. The objective of the puzzle is to move the entire stack to another rod, obeying the following simple rules:

1. Only one disk can be moved at a time.
2. Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack i.e., a disk can only be moved if it is the uppermost disk on a stack.
3. No disk may be placed on top of a smaller disk.



a. JAVA PROGRAM FOR TOWER HANOI

```
1  import java.util.*;
2  class Run
3  {
4      // Java recursive function to solve tower of hanoi puzzle
5      static void towerOfHanoi(int n, char from_rod, char to_rod, char aux_rod)
6      {
7          if (n == 1) //if there is only 1 disc//
8          {
9              System.out.println("Move disk 1 from rod " + from_rod + " to rod " + to_rod);
10             return;
11         }
12         towerOfHanoi(n-1, from_rod, aux_rod, to_rod); //other wise recursive method will
13         //be called//
14         System.out.println("Move disk " + n + " from rod " + from_rod + " to rod " +
15         to_rod);
16         towerOfHanoi(n-1, aux_rod, to_rod, from_rod);
17     }
18     // Driver method
19     public static void main(String args[])
20     {
21         Scanner sc=new Scanner(System.in);
22         System.out.print("Enter Number of Discs: ");
23         int n=sc.nextInt();
24         towerOfHanoi(n, 'A', 'C', 'B');
25         // A, B and C are names of rods
26     }
```

Output:

Move disk 1 from rod A to rod B

Move disk 2 from rod A to rod C

Move disk 1 from rod B to rod C

Move disk 3 from rod A to rod B

Move disk 1 from rod C to rod A

Move disk 2 from rod C to rod B

Move disk 1 from rod A to rod B

Move disk 4 from rod A to rod C

Move disk 1 from rod B to rod C

Move disk 2 from rod B to rod A

Move disk 1 from rod C to rod A

Move disk 3 from rod B to rod C

Move disk 1 from rod A to rod B

Move disk 2 from rod A to rod C

Move disk 1 from rod B to rod C

2.9 String Reversal Using Stack

b. JAVA PROGRAM FOR STRING REVERSAL

String reversal using Stack

```
1  import java.util.*;
2  class StackClass
3  {
4      static int top ; //Created a TOP integer//
5      char myStack[]; //Declared a Character array to store string//
6      static int capacity; //Created a Capacity Integer//
7      String rev1=""; //Created a String variable rev1 with null value//
8      StackClass(int size) //Created a parameterized Constructor//
9      {
10         myStack = new char[size]; //the size entered by user will be Array Size//
11         capacity = size; //capacity will be the same as size entered by user//
12         top = -1; //top value will be -1//
13     }
14     void push(char x) //Created a Push Method to Insert a Character//
15     {
16         top++; //Top pointer will be increased//
17         myStack[top] = x; //Character will take place in the array//
18     }
19     char pop() //Created a Pop Method to Remove a element from Stack//
20     {
21         top--; //Top pointer will be decreased//
22         return myStack[top+1];
23     }
24     String reverse(String str) //Created A method to Reverse A String//
25     {
26         int i;
27         for(i = 0; i < str.length(); i++)
28         {
29             push(str.charAt(i));
30         }
31
32         for(i = 0; i < str.length(); i++)
33         {
34             rev1 = rev1 + pop();
35         }
36         return rev1;
37     }
38 }
39 class ReverseString
40 {
41     public static void main(String args[])
42     {
43         System.out.println("Enter the string you want to reverse - ");
44         Scanner sc = new Scanner(System.in);
45         String rev = sc.nextLine();
46         StackClass r = new StackClass(rev.length());
47         String myRevString = r.reverse(rev);
48         System.out.println(myRevString);
49     }
50 }
```

2.10 Find Binary of The Given Decimal Number Using Stack

c. JAVA PROGRAM TO FIND BINARY FROM DECIMAL

Find binary of the given Decimal number using Stack

```
1  import java.util.*;
2  class StackClass
3  {
4      int top = -1; //Created a top integer with value -1//
5      int myStack[] = new int[100]; //created an Array with a size//
6      void push(int x) //Created a method to push an integer//
7      {
8          top++;
9          myStack[top] = x;
10     }
11     void pop() //Created a method to pop the element//
12     {
13         top--;
14         System.out.print(myStack[top+1]);
15     }
16     void findBin(int n) //Created a method to find Binary Number//
17     {
18         while(n>0) //While n is positive//
19         {
20             push(n%2); //recall push method withn value of n%2//
21             n=n/2; //new value of n will be n/2//
22         }
23         while(top != -1)//while top value is not equal to -1//
24         {
25             pop(); //pop method will be recalled/
26         }
27     }
28 }
29 class DesitoBin
30 {
31     public static void main(String args[])
32     {
33         System.out.println("Enter the number - ");
34         Scanner sc = new Scanner(System.in);
35         int num = sc.nextInt();
36         StackClass s = new StackClass();
37         s.findBin(num);
38     }
39 }
```

